EXAMBHAI

A PROJECT REPORT FOR

AN EXAM PREPARATION SITE FOR MULTIPLE COURSES

BY

MAYANK KUMAR PODDAR

23f3004197 | Modern Application Development – II | IIT Madras

Author:

Name: Mayank Kumar Poddar
Email: 23f3004197@ds.study.iitm.ac.in
Website: https://mynkpdr.github.io

I am currently in diploma level of the BS degree in Data Science from IIT Madras. My passion for computers and technology began in my childhood, and this enthusiasm has driven me ever since. I am dedicated to pushing my boundaries and continuously expanding my knowledge to achieve impactful and meaningful growth.

Project Description:

This project is from the Modern Application Development - II course which is offered by IITM BS Degree during the Diploma in Programming. It is a multi-user app that acts as an exam preparation site for multiple courses. The platform provides both practice and exam modes for students to test their knowledge across various subjects and allows admins to create, manage, and analyse the quiz.

How I approached the problem statement?

Having just completed my MAD-I project in the previous term, I had already gained significant experience in Flask, database design, and project management. This gave me a head start on this project for the backend, but since Vue.js was new to me, I had to go through another learning curve. Just as I had done for my MAD-I project, I relied on various resources like YouTube tutorials, documentation, AI and also StackOverflow and GitHub for any errors that wasn't easy to solve. I also dedicated more time to the project than other courses, ensuring I grasped the necessary concepts quickly.

The initial phase was quite challenging. It was a mix of brainstorming and decision making. I had to figure out critical aspects such as:

- When can a student attempt the quiz?
- Should there be unlimited attempts or just a single attempt?
- When should the results be displayed?
- What types of questions should be included?
- What should the logic behind the charts?

These questions constantly occupied my mind, even in my dreams. To tackle the project efficiently, I started by implementing a proper login system. Once that was in place, I moved on to setting up different routes: admin routes, quiz routes, student routes, static pages and finally backend jobs. As the final submission deadlines approached, I thought of submitting my project in the second batch which had deadline of 15th March. So, I took the opportunity to add some extra functionalities.

Technologies used:

Below are some of the important frontend libraries that I've used in this project.

- **Vue.js 3 (Composition API):** JavaScript framework for building user interfaces
- **Bootstrap 5:** CSS framework for responsive design
- **Chart.js:** JavaScript library for data visualization
- **Axios:** HTTP client for API requests
- **Vue Router:** Navigation management
- **Notivue:** Notification system
- **Pinia:** State management library for Vue 3
- **Simple Datatables:** Lightweight JavaScript HTML table library

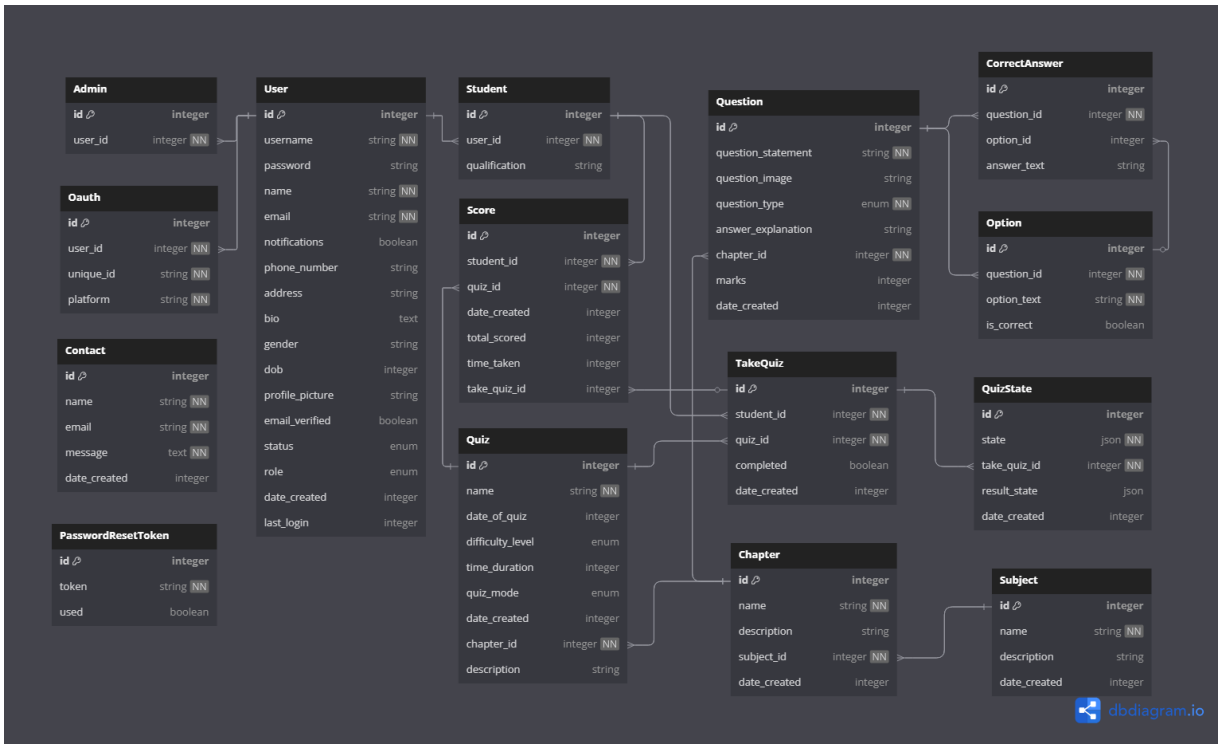
And for the backend:

- **Flask:** Python web framework
- **Flask-RESTx:** API development extension for Flask
- **Flask-SQLAlchemy:** ORM for database interaction
- **Flask-JWT-Extended:** Authentication handling with JWT
- **Flask-Mail:** Email functionality
- **Flask-Migrate:** Database migration
- **Flask-Bcrypt:** Password hashing
- **Flask-Caching:** Response caching
- **Celery:** Distributed task queue
- **Redis:** Message broker for Celery and caching

In addition to these, several other modules and libraries contribute to the functionality and security of this web application.

Database Schema Design:

The application utilizes a relational database with 15 key models, ensuring structured data management. It features a User model for authentication and role-based access, a Quiz model for metadata and settings, and a Question model supporting various question types. The TakeQuiz and QuizState models track student attempts and responses, while Chapter and Subject models organize content hierarchically and more other important tables. Together, these models provide a proper solution to a quiz practice website with RBAC. Below is the database schema made using dbdiagram.io:



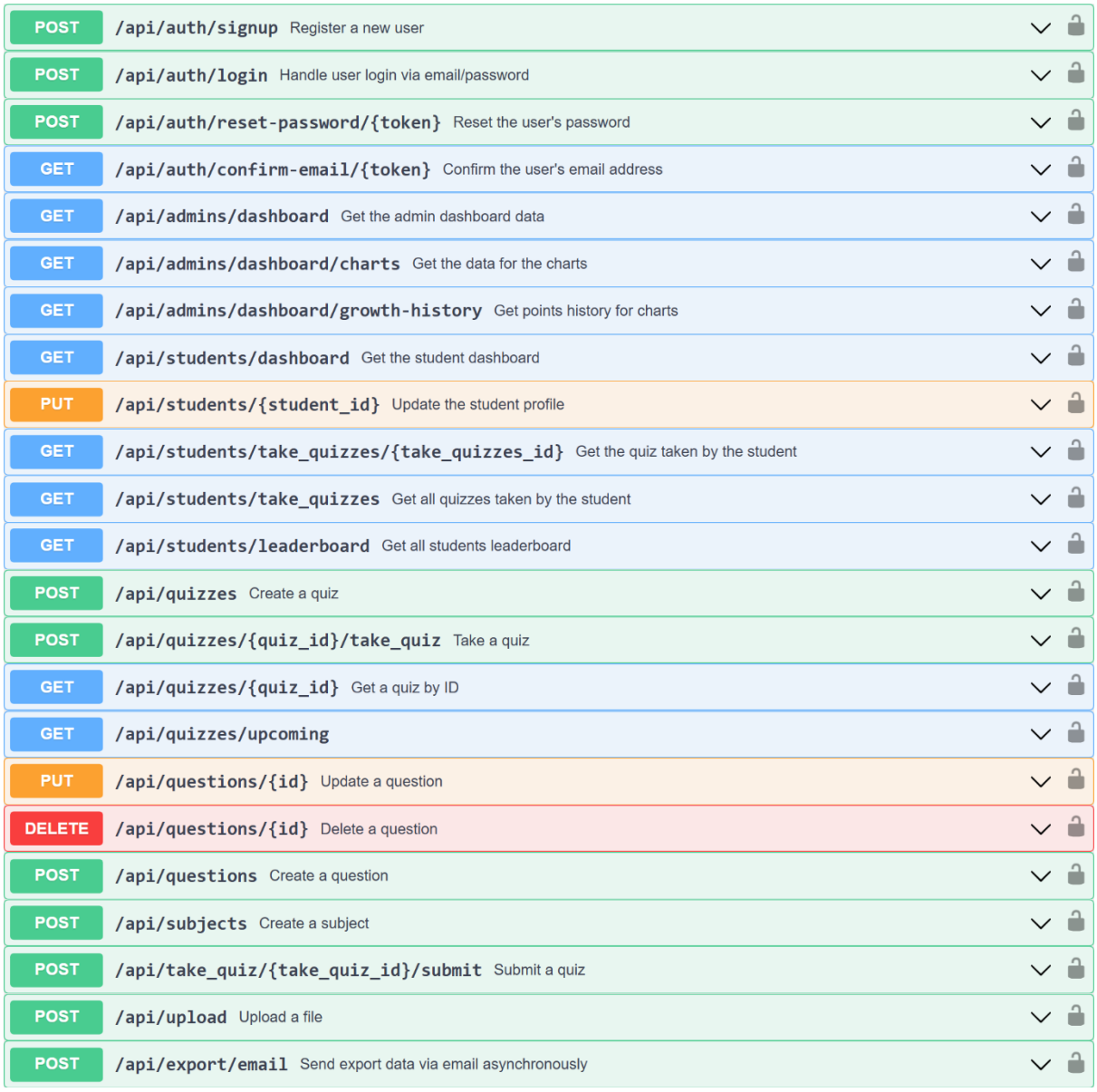
Architecture and Features:

- **Frontend-Backend Separation:**
  - Vue.js frontend consuming REST API
  - Flask backend providing RESTful endpoints
  - JWT authentication for secure communication
- **Task Processing Pipeline:**
  - Celery workers for asynchronous tasks
  - Redis as message broker
  - Background processing for reports and emails
  - Scheduled tasks for notifications and analytics
- **Caching**
  - Redis cache for API response caching
  - Optimized dashboard data retrieval

<table>
<tr><th>STUDENT FEATURES</th><th>ADMIN FEATURES</th></tr>
<tr><td>1. Comprehensive Dashboard:
- Performance analytics
- Subject-wise average scores
- Time vs. score analysis
- Points tracking and history
- Comparison between exam and practice performance</td><td>1. Dashboard Analytics:
- Student activity monitoring
- Quiz completion statistics
- Performance metrics
- Difficulty level distribution
- Growth tracking</td></tr>
<tr><td>2. Quiz Experience:
- Multiple question types (MCQ, MSQ, Numerical)
- Question navigation and review marking
- Scientific calculator for complex problems in exams
- Time tracking and automatic submission
- Detailed results and feedback</td><td>2. Content and Student Management:
- Quiz creation and scheduling
- Questions management
- Lock features while adding question for time efficiency.
- Subject and chapter organization
- CRUD the quizzes, subjects, chapters and the questions.
- Extensive filter and sorting options for all the tables.</td></tr>
<tr><td>3. Dual Quiz Interfaces:
- Exam Mode: Formal, assessment with strict rules similar to the IITM BS Degree quiz exams.
- Practice Mode: Self-paced learning with immediate result and feedback.</td><td>3. Student Management:
- Student’s quiz history and analytics tracking
- Student profile management
- Student quiz summary
- Suspend and block the student</td></tr>
<tr><td colspan="2">4. Data Exports:
- CSV and JSON exports with direct download and email support.
5. Night Mode Toggle:
- Switch between light mode and night mode.</td></tr>
</table>

API Design and Endpoints:

There are more than 45 API endpoints created using Flask-RESTX. Some of them are:



Video:

https://drive.google.com/drive/folders/1cCW1yWMo1lRE4uOeRBCo27NoUT6eoS7e